GENERATIVE AI & LARGE LANGUAGE MODELS

Complete Exam Answers

Long-answer questions • Short-answer questions • Definitions • Examples

Study guide: Written in clear, exam-ready language with definitions, working steps, comparison tables, examples, and conclusions.

Topics include GPT, BERT, LLaMA, Claude, PaLM, fine-tuning, in-context learning, prompt templates, prompt chaining, sampling, bias and fairness, deepfakes, and misinformation.

Part A — Long-answer questions

1. Differentiate between fine-tuning and in-context learning

Introduction: Fine-tuning and in-context learning are methods for adapting a pre-trained Large Language Model (LLM) to a task. The key difference is that fine-tuning updates model parameters, while in-context learning uses instructions or examples in the prompt without updating those parameters.

A. Fine-tuning

Fine-tuning is the process of training an already pre-trained model further on a specific dataset so that it performs better on a particular task. For example, a general language model can be trained on labelled customer-feedback examples to classify feedback as positive, negative, or neutral.

Steps: (1) Select a pre-trained model. (2) Prepare task-specific data. (3) Train the model. (4) Evaluate its performance. (5) Deploy the adapted model.

B. In-context learning

In-context learning provides instructions and examples directly in the prompt. The model uses these examples to infer the requested pattern without changing its learned parameters.

Example: "The lecture was excellent → Positive; The lecture was boring → Negative; The class was useful → ?" The expected answer is "Positive."

Feature	Fine-tuning	In-context learning
Meaning	Further trains a pre-trained model	Provides examples through a prompt
Parameters	Updated during training	Not updated
Data	Usually requires a training dataset	Uses prompt examples, if needed
Setup cost	Often higher due to training	Usually lower setup cost
Setup time	Requires training and evaluation	Can be used immediately
Best suited for	Repeated, specialized tasks	Quick adaptation and experimentation
Example	Train a model for feedback classification	Show examples of feedback classification in a prompt

Advantages and limitations: Fine-tuning can provide persistent task-specific behaviour but needs training resources and can overfit. In-context learning is flexible and easy to test, but prompt length and example quality limit it.

Conclusion: Fine-tuning is useful for persistent task-specific adaptation; in-context learning is useful when quick flexibility without retraining is desired.

2. Differentiate between fine-tuning and instruction tuning

Fine-tuning is the broad process of adapting a pre-trained model using additional data. **Instruction tuning** is a type of fine-tuning in which the model is trained on instructions paired with suitable responses.

Example: Fine-tuning a model on banking conversations helps it handle banking terminology. Instruction tuning teaches it to follow requests such as "Explain the difference between a savings account and a current account in simple language."

Feature	Fine-tuning	Instruction tuning
Definition	Further training on task- or domain-specific data	Training on instruction-response examples
Purpose	Adapt performance or behaviour to a task	Improve instruction following

Feature	Fine-tuning	Instruction tuning
Data	Text, labels, domain examples, or other formats	Instructions and expected responses
Scope	Broad category of training methods	Specific form of fine-tuning
Outcome	Better target-task performance	Better execution of user instructions

Conclusion: All instruction tuning is a form of fine-tuning, but not all fine-tuning is instruction tuning.

3. Demonstrate how GPT and BERT can be applied to text generation and text classification

GPT: Generative Pre-trained Transformer is commonly a decoder-only Transformer that predicts the next token using preceding context.

GPT working process: (1) Receive a prompt. (2) Tokenize the text. (3) Convert tokens to embeddings with positional information. (4) Process context using masked self-attention and feed-forward layers. (5) Calculate next-token probabilities. (6) Select a token using a decoding strategy. (7) Repeat until generation ends.

Example: Input: “Artificial intelligence is”. GPT may continue with “transforming education, healthcare, and industry.” Applications include chatbots, code generation, summarization, translation, and drafting.

BERT: Bidirectional Encoder Representations from Transformers uses an encoder-only architecture. Its original pre-training uses masked language modelling, learning representations from both left and right context.

BERT classification process: (1) Tokenize the input. (2) Add special tokens such as [CLS] and [SEP], where appropriate. (3) Process the sequence through bidirectional encoder layers. (4) Use a representation such as [CLS] with a task-specific classification layer. (5) Predict a label.

Example: “The movie was wonderful and exciting” → Positive; “The movie was slow and disappointing” → Negative.

Feature	GPT	BERT
Architecture	Decoder-only Transformer	Encoder-only Transformer
Typical objective	Autoregressive next-token prediction	Masked language modelling in original pre-training
Context	Preceding tokens for standard generation	Left and right context
Common use	Text generation	Text classification and understanding
Output example	Generated essay	Spam / not-spam label

Conclusion: GPT is particularly suitable for generating text, while BERT is particularly useful for understanding and classifying text. Both can be adapted to other tasks.

4. Explain the architectures and major features of GPT, BERT, LLaMA, Claude, and PaLM

1. GPT

Architecture: Decoder-only Transformer. Text is tokenized and embedded; masked self-attention processes preceding tokens; feed-forward layers transform representations; the model predicts and generates the next token repeatedly.

Features: Text and code generation, conversational question answering, summarization, translation, and instruction following in instruction-tuned versions. **Example:** Generating Python code from a natural-language description.

2. BERT

Architecture: Encoder-only Transformer. It processes tokens with bidirectional self-attention and generates contextual representations. A task-specific layer can use these representations for classification or other downstream

tasks.

Features: Sentiment analysis, text classification, named-entity recognition, and extractive question answering. **Example:** Classifying an email as spam or legitimate.

3. LLaMA

Full name: Large Language Model Meta AI. **Architecture:** Decoder-only Transformer. LLaMA is a family of generative models developed by Meta; architectural details vary by generation.

Features: Text and code generation, question answering, task adaptation, and deployment through supported inference frameworks. Some versions can run locally when hardware and licensing permit. **Example:** A locally hosted assistant that answers questions about college notes.

4. Claude

Claude is a family of AI models developed by Anthropic. It uses proprietary Transformer-based language-model technology; not every internal architectural detail is public.

Features: Conversational assistance, document summarization, coding, reasoning, and instruction following. Anthropic has researched safety-oriented approaches including Constitutional AI. **Example:** Summarizing a project report and providing recommendations.

5. PaLM

PaLM stands for Pathways Language Model and was developed by Google. It is a decoder-only Transformer language model.

Features: Language generation and understanding, few-shot learning, reasoning, and code-related tasks. **Example:** Providing examples of arithmetic word problems and asking the model to solve a similar one. PaLM is historically important; newer Google model families have superseded it in many products.

Model	Architecture	Primary strengths
GPT	Decoder-only Transformer	Text generation and conversation
BERT	Encoder-only Transformer	Text understanding and classification
LLaMA	Decoder-only Transformer	Generation, adaptation, deployment flexibility
Claude	Proprietary Transformer-based family	Conversation, coding, document analysis, reasoning
PaLM	Decoder-only Transformer	Language generation, few-shot learning, reasoning

Conclusion: GPT, LLaMA, Claude, and PaLM are generative model families, while BERT is especially associated with bidirectional text understanding. Architecture and training help determine each model's strengths.

5. Explain the role of prompt templates in LangChain and LlamaIndex for task automation

Definition: A prompt template is a reusable structure containing instructions and placeholders for dynamic information. Example: "Summarize the following text in {number} sentences: {text}."

A. LangChain

LangChain helps developers build LLM applications, workflows, tool integrations, and chains. Prompt templates support reusability, dynamic input insertion, consistent instructions, model integration, and output parsing.

Python example:

```
from langchain_core.prompts import PromptTemplate template = PromptTemplate.from_template( "Explain {topic} in {sentences} sentences." ) prompt = template.format( topic="Artificial Intelligence", sentences=3 ) print(prompt)
```

Output: "Explain Artificial Intelligence in 3 sentences."

B. LlamaIndex

LlamaIndex connects language models with external data such as PDFs, documents, and databases. Prompt templates guide document querying, retrieval-augmented generation (RAG), summarization, and question answering.

Python example:

```
from llama_index.core import PromptTemplate template = PromptTemplate( "Answer using the context below.\n" "Context: {context_str}\n" "Question: {query_str}\n" "Answer:" ) prompt = template.format( context_str="RAG retrieves relevant documents.", query_str="What is RAG?" ) print(prompt)
```

Exact APIs can vary by framework version and workflow.

How templates automate tasks

- Define a reusable prompt with placeholders.
- Insert user input and retrieved data.
- Send the completed prompt to the language model.
- Process the output and continue the workflow.

Feature	LangChain	LlamaIndex
Main focus	LLM application workflows and integrations	Connecting LLMs with data
Prompt use	Chains, tools, agents, model interactions	Retrieval, document querying, response synthesis
Example	Generate customer-support replies	Answer questions from college PDFs

Conclusion: Prompt templates improve consistency, reusability, and automation. LangChain often uses them to orchestrate applications; LlamaIndex often uses them in document-based retrieval and answering.

6. Explain prompt chaining with a suitable example

Definition: Prompt chaining connects multiple prompts or LLM operations so the output from one step becomes input to the next. A complex task is split into smaller tasks.

Example: Automatic college project report.

1. Topic analysis: identify the problem, objectives, and scope.

2. Outline generation: create report chapters and headings from the analysis.

3. Section writing: draft the introduction, methodology, and proposed solution.

4. Review and formatting: check consistency, missing sections, and formatting.

5. Final output: produce the report.

Simple Python demonstration:

```
def analyze_topic(topic): return f"Problem and objectives identified for: {topic}" def create_outline(analysis): return f"Report outline based on: {analysis}" def write_report(outline): return f"Report draft based on: {outline}" topic = "AI-based smart traffic management" analysis = analyze_topic(topic) outline = create_outline(analysis) report = write_report(outline) print(report)
```

This example shows the data flow with ordinary Python functions; a real LLM workflow would call a language model inside each stage.

Advantages: Breaks down complex tasks, simplifies testing and debugging, gives each prompt a clear responsibility, and enables intermediate-output inspection. **Limitations:** Early errors can propagate, multiple calls increase cost and latency, and outputs need validation.

Conclusion: Prompt chaining connects smaller AI tasks into a structured workflow that is easier to monitor and improve.

7. Differentiate between Top-k and Top-p sampling

Introduction: Top-k and Top-p are decoding strategies that control which candidate tokens can be selected next during generation.

A. Top-k sampling

Top-k retains the k tokens with the highest predicted probabilities. The model samples from those candidates after normalizing their probabilities. If the probabilities are Apple 0.40, Mango 0.30, Banana 0.15, Orange 0.10, and Grape 0.05, then k = 3 retains Apple, Mango, and Banana.

B. Top-p sampling

Top-p, also called nucleus sampling, retains the smallest set of highest-probability tokens whose cumulative probability reaches or exceeds threshold p. In the same example, p = 0.70 retains Apple and Mango because their cumulative probability is 0.70. With p = 0.80, Banana is also included, taking the cumulative probability to 0.85.

Feature	Top-k	Top-p
Selection rule	Fixed number of top candidates	Cumulative probability threshold
Candidate count	Fixed k, subject to available vocabulary	Variable
Control	Number of candidates	Probability mass
Example	k = 3	p = 0.9

Both methods can be combined with temperature and other decoding controls depending on the implementation. **Conclusion:** Top-k fixes the number of candidates; Top-p adapts the candidate set to the probability distribution.

8. Explain major challenges of bias and fairness in LLMs and generative systems

Introduction: Bias refers to systematic patterns in a model's outputs that can produce inaccurate, unfair, or unequal outcomes. Fairness means avoiding unjustified discrimination and unfair treatment.

Major challenges

1. Training-data bias: If training data contains historical discrimination or stereotypes, a model may reproduce them. Example: associating leadership mainly with men.

2. Representation bias: Some languages, cultures, or demographic groups may be underrepresented. Example: a chatbot may understand common English better than local dialects.

3. Stereotyping: The model may make unjustified assumptions about people based on group characteristics. Example: portraying some occupations as suitable only for one gender.

4. Algorithmic bias: Model design, data selection, objectives, thresholds, or deployment decisions can introduce bias. Example: a recruitment system reproducing discriminatory patterns in historical hiring data.

5. Evaluation bias: Testing on a narrow population can hide weaknesses elsewhere. Example: speech recognition tested mainly on one accent may perform worse for other accents.

6. Bias in generated images: Image systems can repeat stereotypes or underrepresent groups. Example: a generic scientist prompt producing limited demographic representation.

7. Feedback loops: Biased outputs used as new data or inputs to later decisions can reinforce existing patterns. Example: a recommendation system repeatedly promoting only a narrow range of creators.

Impact

- Unfair hiring and educational decisions.
- Unequal access to services.
- Reinforcement of stereotypes.
- Reduced trust in AI.
- Incorrect or offensive content.
- Disproportionate harm to underrepresented communities.

Ways to improve fairness

- Use diverse, representative datasets.
- Identify and reduce harmful stereotypes during data preparation and training.
- Evaluate performance across relevant groups.
- Conduct regular audits and real-world testing.
- Use human review for high-impact decisions.
- Monitor systems after deployment and provide ways to report errors.

Conclusion: LLMs may reproduce biases in data and system design. Diverse data, systematic evaluation, transparency, and human oversight help reduce unfair outcomes.

Alternative wording: Analyze the impact of bias on generative AI outputs

Text generation may produce stereotypes or unfair recommendations; image generation may offer limited or stereotypical representation; speech systems may perform unequally across accents; and decision-support tools may produce biased summaries or recommendations. The impact depends on the application and how the output is used.

9. Explain how deepfakes can be used to spread misinformation

Definition: A deepfake is synthetic or manipulated media—such as video, images, or audio—created using AI to make a person appear to say or do something that did not actually happen. Deepfakes can have legitimate uses, but can also be misused.

Ways deepfakes spread misinformation

1. Fabricated political statements: A manipulated video or audio clip falsely depicts a public figure making a controversial statement. Viewers may believe it is genuine.

2. Fake news videos: A manipulated video may be presented as evidence of an event that never occurred, creating confusion or public fear.

3. Voice impersonation: Synthetic audio may imitate a speaker and make a fabricated message seem authentic.

4. Fake celebrity endorsements: A generated video may falsely show a celebrity endorsing a product, potentially deceiving consumers.

5. Fraudulent recordings: Impersonated voices may be used in scams involving family members, businesses, or institutions.

How misinformation spreads

1. AI-generated or manipulated media

2. A false or misleading caption is added

3. Content is shared through social media or messaging apps

4. People believe or reshare it

5. Misinformation spreads and may cause harm

Detection and prevention

- Verify claims through reliable news organisations and official sources.
- Check the original source and publication date.
- Seek independent confirmation before sharing.
- Use media-forensics tools where appropriate.
- Check provenance information and authenticity credentials when available.
- Improve public awareness of synthetic media.

- Use appropriate labelling and safeguards for AI-generated content.

No single visual clue or detection tool can reliably identify every deepfake. Verification should consider the source and supporting evidence as well as the media.

Conclusion: Deepfakes can make false information appear authentic. Source verification, media literacy, technical detection, and responsible AI practices help reduce misinformation.

Part B — Important 2-mark questions

1. Define GPT.

GPT stands for Generative Pre-trained Transformer. It is a type of language model commonly based on a decoder-only Transformer that generates text by predicting the next token from preceding context. Example: generating answers, summaries, and code.

2. What is a pre-trained Large Language Model (LLM)?

A pre-trained LLM is an AI model trained on a large collection of text to learn language patterns and context before being adapted to specific tasks. Example: adapting a model for question answering.

3. What is Top-k sampling?

Top-k sampling restricts the next-token candidates to the k tokens with the highest predicted probabilities, then samples from those candidates. Example: k = 3 retains the three highest-probability candidates.

4. What is a prompt template?

A prompt template is a reusable structure with instructions and placeholders filled with dynamic input. Example: “Explain {topic} in simple words.”

5. What is algorithmic bias?

Algorithmic bias is a systematic pattern in an algorithm’s outputs or decisions that can produce unfair outcomes for individuals or groups. Example: a recruitment algorithm disadvantaging qualified applicants due to biased historical data.

6. Define deepfakes.

Deepfakes are AI-generated or AI-manipulated images, videos, or audio that make a person appear to say or do something that did not actually happen. Example: a fabricated video of a public figure.

7. List any two types of bias in LLMs.

(1) Representation bias: some groups, languages, or viewpoints are inadequately represented. (2) Stereotyping bias: a model reproduces unfair assumptions about people based on group characteristics.

8. What is PaLM?

PaLM stands for Pathways Language Model. It is a large language model developed by Google using a decoder-only Transformer architecture for language generation, understanding, and reasoning.

9. What is fine-tuning in LLMs?

Fine-tuning is further training a pre-trained language model on task-specific or domain-specific data to improve performance on that task. Example: training on labelled customer-support conversations.

10. Define Top-p sampling.

Top-p, or nucleus sampling, retains the smallest set of high-probability tokens whose cumulative probability reaches or exceeds a threshold p, then samples from that set.

11. What is agentic prompting?

Agentic prompting guides an AI system to pursue a goal through steps such as planning, using tools, observing results, and deciding what to do next. Example: searching documentation and preparing a summary.

12. What is voice cloning?

Voice cloning is an AI technique that creates synthetic speech resembling a particular speaker using voice samples or other training information. Applications include accessibility and authorised dubbing.

13. What is fairness evaluation in LLMs?

Fairness evaluation assesses whether an LLM produces systematically different or unfair outputs for different demographic groups. Example: comparing answer quality across dialects.

14. Define misinformation.

Misinformation is false or inaccurate information shared regardless of whether the person sharing it intends to deceive others. Example: sharing an incorrect news report believing it is true.

Part C — Additional quick-revision terms

Term	Meaning
LLM	Large Language Model trained to process and generate language.
Transformer	Neural-network architecture based on attention mechanisms.
Self-attention	Mechanism that helps a model identify relationships between tokens.
BERT	Encoder-based Transformer associated with bidirectional text understanding.
LLaMA	Meta's family of generative language models.
Claude	Anthropic's family of AI language models.
Fine-tuning	Further training a pre-trained model on additional data.
Instruction tuning	Fine-tuning on instructions and expected responses.
In-context learning	Using examples in a prompt without updating model parameters.
Prompt chaining	Connecting prompts so one step's output feeds into the next.
Prompt engineering	Designing and refining prompts to obtain useful outputs.
RAG	Retrieval-Augmented Generation: retrieving external information to help an LLM answer.
Deepfake	AI-generated or manipulated media that imitates people or events.
Algorithmic bias	Systematic patterns in algorithmic outputs that may lead to unfair results.
Fairness	Avoiding unjustified discrimination or unequal outcomes.
Voice cloning	Creating synthetic speech resembling a particular speaker.

Last-minute exam preparation strategy

1. Fine-tuning vs. in-context learning

2. Fine-tuning vs. instruction tuning

3. GPT vs. BERT

4. Architectures and features of GPT, BERT, LLaMA, Claude, and PaLM

5. Prompt templates in LangChain and LlamaIndex

6. Prompt chaining

7. Top-k vs. Top-p sampling

8. Bias and fairness in LLMs

9. Deepfakes and misinformation

For long answers, remember the definition, working steps, comparison points, one real-world example, and conclusion. For 2-mark questions, learn a clear definition and one example where appropriate.